

Cuestionario clases 3 y 4

1. a. Explique la semántica de un semáforo. ¿Que diferencia hay entre los semáforos generales y los binarios?. ¿Como se representan las operaciones de cada uno con await?

- Los semáforos tienen dos instrucciones: P() y V(). La primera instrucción se usa para demorar al proceso hasta que ocurra un evento, y la segunda se utiliza para señalar la ocurrencia de un evento. A efectos prácticos, la primera instrucción decrementa el valor del semáforo, y la segunda lo incrementa
- La diferencia reside en los valores que pueden tomar los semáforos. Un semáforo binario solo puede tener un valor 0 o 1, el general cualquier valor entero no negativo.
- `<await (free > 0) free = free - 1; SC; <free = free + 1>; SNC`

1. b. Indique los posibles valores finales de x en el siguiente programa

```
int x = 4;
sem s1 = 1, s2 = 0;
co P(s1); x = x * x ; V(s1);
  // P(s2); P(s1); x = x * 3; V(s1);
  // P(s1); x = x - 2; V(s2); V(s1);
oc
```

- $x = 16 \rightarrow x = 14 \rightarrow x = 42$
- $x = 2 \rightarrow x = 8 \rightarrow x = 36$
- $x = 2 \rightarrow x = 4 \rightarrow x = 12$

2. Desarrolle utilizando semáforos una solución centralizada al problema de los filósofos, con un administrador único de los tenedores, y posiciones libres para los filósofos (es decir, cada

filósofo puede comer en cualquier posición siempre que tenga los dos tenedores correspondientes).

```
sem tenedores = 5;
sem puedeComer = 2;
sem avisarTermino = 0;

process Filosofo [i: 0..4]
{ while (true)
    P(puedeComer);
    { P(tenedores);
      P(tenedores);
      V(avisarTermino);
    }
}

process Administrador
{ while (true)
    { P(avisarTermino);
      V(tenedores);
      V(tenedores);
      V(puedeComer);
    }
}
```

3. Describa la técnica de Passing the Baton. ¿Cuál es su utilidad en la resolución de problemas mediante semáforos?

La técnica Passing the Baton permite resolver problemas de concurrencia donde varios procesos deben coordinar entre ellos el acceso a un recurso compartido, haciendo exclusión mutua cada uno. Es una técnica general para implementar sentencias await. Cuando un proceso está dentro de una SC mantiene el baton, que significa "permiso para ejecutar". Cuando el proceso llega a un signal, pasa el batón (control) a otro proceso). Si ningún otro proceso está esperando por el baton, el baton se libera para que lo tome el próximo proceso que trata de entrar.

4. Modifique las soluciones de Lectores-Escritores con semáforos de modo de no permitir más de 10 lectores

simultáneos en la BD y además que no se admita el ingreso a más lectores cuando hay escritores esperando

```
int nr = 0, nw = 0, dr = 0, dw = 0; sem e = 1, r = 0, w = 0;
```

```
process Lector [i = 1 to M]
```

```
{ while(true)
  { P(e);
    if (nw > 0) {dr = dr+1; V(e); P(r); }
    nr = nr + 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    else V(e);
    lee la BD;
    P(e);
    nr = nr - 1;
    if (nr == 0 and dw > 0)
      {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

```
process Escritor [j = 1 to N]
```

```
{ while(true)
  { P(e);
    if (nr > 0 or nw > 0)
      {dw=dw+1; V(e); P(w);}
    nw = nw + 1;
    V(e);
    escribe la BD;
    P(e);
    nw = nw - 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    elseif (dw > 0) {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

```
}  
}
```

Se debe reescribir en la primera condición de los lectores así

if (nw > 0 || dw > 0) para que no se admita el ingreso de lectores si hay escritores esperando.

Para no permitir más de 10 lectores simultáneos, se puede usar un semáforo contador

de recursos. El código de los lectores quedaría

sem mutexR = 10; // Semaforo contador de lectores en la BD.

```
process Lector [i = 1 to M] {
```

```
while(true) {
```

```
  P(mutexR)
```

```
  P(e);
```

```
  if (nw > 0 || dw > 0) { // Si hay escritores esperando o en BD se duerme
```

```
    dr = dr+1;
```

```
    V(e);
```

```
    P(r);
```

```
  }
```

```
  nr = nr + 1;
```

```
  if (dr > 0) {
```

```
    dr = dr - 1;
```

```
    V(r);
```

```
  }
```

```
  else V(e);
```

```
  lee la BD;
```

```
  P(e);
```

```
  nr = nr - 1;
```

```
  if (nr == 0 and dw > 0) {
```

```
    dw = dw - 1;
```

```
    V(w);
```

```
  }
```

```
  else V(e);
```

```
  V(mutexR)
```

```
}
```

```
}
```

5. Describa el funcionamiento de los monitores como herramienta de sincronización. Como se realiza la comunicación y la sincronización por condición entre procesos en esta herramienta

módulos de programa con más estructura, y que pueden ser implementados tan eficientemente como los semáforos. Los monitores se asemejan a objetos, ya que encapsulan las representaciones de recursos y brindan un conjunto de operaciones que son los únicos medios para manipular esos recursos. además, contiene variables que almacenan el estado del recurso y procedimientos que implementan las operaciones sobre él

Exclusión Mutua  implícita asegurando que los procedimientos en el mismo monitor no ejecutan concurrentemente. Sincronización por Condición  explícita con variables condición

6. ¿Qué diferencias existen entre las disciplinas de señalización "Signal and wait" y "Signal and continue"?

En el SIGNAL AND CONTINUE, el proceso que hace el signal continua usando el monitor y el proceso despertado pasa a competir por el acceso al monitor, mientras que en el SIGNAL AND WAIT ocurre al revés, este proceso pasa a competir por acceder al monitor, mientras que el proceso despertado pasa a ejecutar dentro del monitor a partir de la instrucción siguiente al wait

7. ¿En qué consiste la técnica de Passing the Condition y cuál es su utilidad en la resolución de problemas con monitores? ¿Qué relación encuentra entre passing the condition y passing the baton?

La técnica Passing the Condition consiste en utilizar una variable para bloquear el acceso a un recurso, y que el proceso que termine de usar la variable se fije primero si hay algún proceso esperando utilizar el recurso para despertarlo y no tener que poner la variable en true, y así liberar el recurso, lo que evita que un proceso que haya estado esperando menos que los otros pueda reclamar el recurso antes que ellos, y así garantizar el orden.

Su utilidad en la resolución de problemas con monitores es que introduce un orden entre los que esperan por un recurso, y evita las demoras innecesarias con el pasaje que se realiza cuando un proceso termina de usar el recurso y le pasa el

acceso a otro. Se parece a la técnica Passing the Baton en que, quien decide cuando utiliza el recurso el próximo proceso es el proceso que está utilizando el recurso actualmente, una vez que deja de usarlo, y hasta entonces los otros procesos, si es que los hay, están dormidos.

8. Desarrolle utilizando monitores una solución centralizada al problema de los filósofos, con un administrador único de los tenedores, y posiciones libres para los filósofos (es decir, cada filósofo puede comer en cualquier posición siempre que tenga los dos tenedores correspondientes).

```
monitor Tenedor [1..5] {
    int[] tenedor = {1, 1, 1, 1, 1}
    cond puedeComer[5];

    procedure Agarrar(int id) {
        int izq = id;
        int der = (id + 1) % 5;

        while (tenedor[izq] == 0 || tenedor[der] == 0) {
            wait(puedeComer[id]);
        }

        tenedor[izq] = 0;
        tenedor[der] = 0;
    }

    procedure Devolver(int id) {
        int izq = id;
        int der = (id + 1) % 5;

        tenedor[izq] = 1;
        tenedor[der] = 1;

        // vecino izquierdo
        int vecinolq = (id + 4) % 5;
```

```

    signal(puedeComer[vecinoIzq]);

    // vecino derecho
    int vecinoDer = (id + 1) % 5;
    signal(puedeComer[vecinoDer]);
}
}

process Filosofo [id: 0..4]
{
    Tenedor.Agarrar(id);
    comer();
    Tenedor.Devolver(id);
}

```

9. Sea la siguiente solución propuesta al problema de alocación SJN:

a) ¿Funciona correctamente con disciplina de señalización Signal and Continue?

No, ya que al no liberarse la variable libre solamente cuando ya hayan terminado de pasar todos los procesos dormidos que estaban esperando, y al ponerse el libre en true ni bien un proceso termina de utilizar el recurso, cuando el proceso que termina de utilizar el recurso utiliza el signal(...) y despierta al otro proceso B, podría pasar tranquilamente que un proceso C llegue a encolarse antes que B, y acceda al recurso antes que B, y pase a reclamar el recurso para sí sin tener que haberse dormido.

b) ¿Funciona correctamente con disciplina de señalización Signal and Wait?

Si, ya que ni bien el proceso actual ejecute el signal(...) el proceso señalado se despertaría y pondría la variable libre en false, y tendría que esperarse a que ella llegue al release() para que se libere, por lo que ningún proceso en el medio podría meterse.

- Bajo esta disciplina, la ejecución del `signal` transfiere el control del monitor y la ejecución **inmediatamente** al proceso que estaba esperando. Como el

proceso despertado retoma su ejecución justo después del `wait` (donde hace `libre = false`), no existe ningún intervalo de tiempo en el que un tercer proceso pueda entrar al monitor y robar el recurso mientras `libre` está en `true`

10. Modifique la solución anterior para el caso de no contar con una instrucción wait con prioridad.

```
monitor SJN {
    bool libre = true;
    colaPri cola;
    cond turno[N];

    procedure request(int id, int tiempo) {
        if (not libre)
            colaPri.insert(id, tiempo);
        wait(turno[id]);
        libre = false;
    }

    procedure release() {
        if not empty(cola)
            int proxId = cola.pop();
            signal(turno[proxId]);
        else
            libre = true;
    }
}
```

- En ese caso deberían utilizarse identificadores, debería agregarse una cola con prioridad y un array de condiciones privadas
- Esta solución funciona para ambas disciplinas de señalización

11. Modifique utilizando monitores las soluciones de Lectores-Escritores de modo de no permitir más de 10 lectores simultáneos

en la BD, y además que no se admita el ingreso a más lectores cuando hay escritores esperando

```
monitor Controlador_RW
{
    int nr = 0, nw = 0, dr = 0, dw = 0;
    cond ok_leer, ok_escribir

    procedure pedido_leer() {
        if (nw > 0 or dw > 0 or nr > 10) {
            dr = dr + 1;
            wait (ok_leer);
        } else nr = nr + 1;
    }

    procedure libera_leer() {
        nr = nr - 1;
        if (nr == 0 and dw > 0) {
            dw = dw - 1;
            signal (ok_escribir);
            nw = nw + 1;
        } else if (dr > 0 and dw == 0) {
            signal(ok_leer);
            dr = dr - 1;
            nr = nr + 1;
        }
    }
}

procedure pedido_escribir () {
    if (nr > 0 OR nw > 0) {
        dw = dw + 1;
        wait (ok_escribir);
    } else nw = nw + 1;
}

procedure libera_escribir() {
```

```

if (dw > 0) {
    dw = dw - 1;
    signal(ok_escribir);
} else {
    nw = nw - 1;
    if (dr > 0) {
        if (dr <= 10) {
            nr = dr;
            dr = 0;
            signal_all(ok_leer);
        } else {
            for (int i = 0; i < 10; i++) {
                nr = nr + 1;
                dr = dr - 1;
                signal(ok_leer);
            }
        }
    }
}
}
}
}
}

```

12. Resuelva con monitores el siguiente problema: tres clases de procesos comparten el acceso a una lista enlazada: searchers, inserters y deleters. Los searchers sólo examinan la lista, y por lo tanto pueden ejecutar concurrentemente unos con otros. Los inserters agregan nuevos ítems al final de la lista; las inserciones deben ser mutuamente exclusivas para evitar insertar dos ítems casi al mismo tiempo. Sin embargo, un insert puede hacerse en paralelo con uno o más searches. Por último, los deleters remueven ítems de cualquier lugar de la lista. A lo sumo un deleter puede acceder la lista a la vez, y el borrado también debe ser mutuamente exclusivo con searches e inserciones.

monitor ManejadorLista

```
{  
    int agregando = 0;  
    int buscando = 0;  
    int borrando = 0;  
  
    cond espera_agregar;  
    cond espera_buscar;  
    cond espera_borrar;  
  
    procedure pedirAgregar() {  
        if (borrando > 0 or agregando > 0)  
            wait(espera_agregar);  
        agregando++;  
    }  
  
    procedure terminarAgregar() {  
        agregando--;  
        if not empty(espera_agregar)  
            signal(espera_agregar);  
        else if not empty(espera_buscar) // Asumo que borrando = 0  
            signal_all(espera_buscando);  
        else if not empty(espera_borrar)  
            signal(espera_borrar);  
    }  
  
    procedure pedirBuscar() {  
        if (borrando > 0)  
            wait(espera_buscar);  
        buscando++;  
    }  
  
    procedure terminarBuscar() {  
        buscando--;  
        if not empty(espera_agregar)
```

```

        signal(espera_agregar);
    else if not empty(espera_borrar) and (buscando == 0)
        signal(espera_borrar);
    }

    procedure pedirBorrar() {
        if (agregando > 0 or buscando > 0 or borrando > 0)
            wait(espera_borrar);
        borrando++;
    }

    procedure terminarBorrar() {
        borrando--;
        if not empty(borrando)
            signal(espera_borrar);
        else if not empty(esperar_agregar)
            signal(espera_agregar);
        if not empty(espera_buscando)
            signal_all(espera_buscando);
    }
}

process searcher [1..N]
{
    while (true) {
        ManejadorLista.pedirBuscar(...);
        wait(random(N)); // Simular que pasa un tiempo fuera del programa
    }
}

process inserter [1..N]
{
    while (true) {
        ManejadorLista.pedirAgregar();
        // Simulo que agrego un elemento, solo quiero reclamar el uso de la l
    }
}

```

```

        ManejadorLista.terminarAgregar();
        wait(random(N));
    }
}

process deleter [1..N]
{
    while (true) {
        ManejadorLista.pedirBorrar();
        // Simulo que borro un elemento, solo quiero reclamar el uso de la lis
ta
        ManejadorLista.terminarBorrar();
        wait(random(N));
    }
}

```

13. El problema del "Puente de una sola vía" (One-Lane Bridge): autos que provienen del Norte y del Sur llegan a un puente con una sola vía. Los autos en la misma dirección pueden atravesar el puente al mismo tiempo, pero no puede haber autos en distintas direcciones sobre el puente.

- a. Desarrolle una solución al problema, modelizando los autos como procesos y sincronizando con un monitor (no es necesario que la solución sea fair ni dar preferencia a ningún tipo de auto).

```

monitor Puente {
    int cruzando_norte = 0;
    int esperando_norte = 0;
    int cruzando_sur = 0;
    int esperando_sur = 0;

    cond espera_norte;
    cond espera_sur;

    procedure CruzarPorNorte() {

```

```

        while (cruzando_sur > 0) {
            esperando_norte++;
            wait(espera_norte);
        }
        cruzando_norte++;
    }

    procedure SalirPorSur() {
        cruzando_norte--;
        if (cruzando_norte == 0) {
            signal_all(espera_sur)
        }
    }

    procedure CruzarPorSur() {
        while (cruzando_norte > 0) {
            esperando_sur++;
            wait(espera_sur);
        }
        cruzando_sur++;
    }

    procedure SalirPorNorte() {
        cruzando_sur--;
        if (cruzando_sur == 0) {
            signal_all(espera_norte)
        }
    }
}

process AutoNorte [1..N] {
    Puente.CruzarPorNorte();
    // Se cruza el puente
    Puente.SalirPorSur();
}

```

```

process AutoSur [1..N] {
    Puente.CruzarPorSur();
    // Se cruza el puente
    Puente.SalirPorNorte();
}

```

- b. **Modifique la solución para asegurar fairness (Pista: los autos podrían obtener turnos)**

```

// Mi solución tiene en cuenta las siguientes cuestiones:
// - Si no hay ningún auto, puede pasar cualquiera
// - Si hay varios autos esperando de un lado, y llevan tiempo
// esperando (CANT_SUC), deberían poder pasar

monitor Puente {

    int cruzando_norte = 0;
    int esperando_norte = 0;
    int cruzaron_seguido_norte = 0;

    int cruzando_sur = 0;
    int esperando_sur = 0;
    int cruzaron_seguido_sur = 0;

    // Número límite elegido de manera arbitraria
    const CANT_SUC = 5;

    cond espera_norte;
    cond espera_sur;

    procedure CruzarPorNorte() {
        esperando_norte++;
        while (cruzando_sur > 0 or (cruzaron_seguido_norte >= CANT_S
UC and esperando_sur > 0)) {
            wait(espera_norte);

```

```

    }
    cruzaron_seguido_sur = 0;
    esperando_norte--;
    cruzando_norte++;
    cruzaron_seguido_norte++;
}

procedure SalirPorSur() {
    cruzando_norte--;
    if (cruzando_norte == 0) {
        if (esperando_sur > 0) {
            signal_all(espera_sur)
        } else {
            // Por si se durmieron autos que vienen del norte
            signal_all(espera_norte);
        }
    }
}

procedure CruzarPorSur() {
    esperando_sur++;
    while (cruzando_norte > 0 or (cruzaron_seguido_sur >= CANT_S
UC and esperando_norte > 0)) {
        wait(espera_sur);
    }
    cruzaron_seguido_norte = 0;
    esperando_sur--;
    cruzando_sur++;
    cruzaron_seguido_sur++;
}

procedure SalirPorNorte() {
    cruzando_sur--;
    if (cruzando_sur == 0) {
        if (esperando_norte > 0) {
            signal_all(espera_norte)

```

```

        } else {
            // Por si se durmieron autos que vienen del sur
            signal_all(espera_sur);
        }
    }
}

process AutoNorte [1..N] {
    Puente.CruzarPorNorte();
    // Se cruza el puente
    Puente.SalirPorSur();
}

process AutoSur [1..N] {
    Puente.CruzarPorSur();
    // Se cruza el puente
    Puente.SalirPorNorte();
}

```

[Consultado a un profesor, solución válida]